INF4067 · PATRONS DE CONSTRUCTION

# Le patron Singleton

Exemple du cours et variante concrète : journaliseur centralisé AppLogger

**Étudiant :** Theodore Wilfried Bitjong

**Matricule :** 23U2331

**Dépôt GitHub :**
https://github.com/inf461-bitjong/singleton

11 novembre 2026

# 1. Présentation du patron

Le Singleton est un patron de création qui garantit qu'une classe ne possède qu'une seule instance et fournit un point d'accès global à celle-ci. Il est adapté lorsqu'une ressource ou un service doit être partagé et centralisé, par exemple un journaliseur ou un gestionnaire de configuration.

## Structure générique

- **Instance statique privée :** conserve l'unique objet créé.
- **Constructeur privé :** empêche les autres classes de créer directement des objets avec `new`.
- **Accesseur statique :** crée l'objet au premier appel puis renvoie toujours la même instance.

> **Choix du modèle générique.** Dans les deux exemples, le modèle Singleton est retenu car le besoin est l'unicité d'un objet accessible depuis plusieurs clients. L'exemple du cours illustre la structure de base et ses opérations ; la variante l'applique à un service transversal réel, le journal d'application.

## Considération de concurrence

La création paresseuse non synchronisée de l'exemple du cours convient à une démonstration séquentielle, mais n'assure pas à elle seule l'unicité lors d'appels simultanés de plusieurs threads. La variante synchronise l'accesseur `getInstance()`, protégeant ainsi la création concurrente de l'instance.

# 2. Exemple vu en cours

La classe `cours.Singleton` conserve deux valeurs entières et expose deux accesseurs surchargés : l'un crée l'objet sans paramètres, l'autre initialise `x` et `y`. Elle fournit aussi des méthodes de somme et de moyenne, un affichage des valeurs et une protection contre le clonage.

## Modélisation

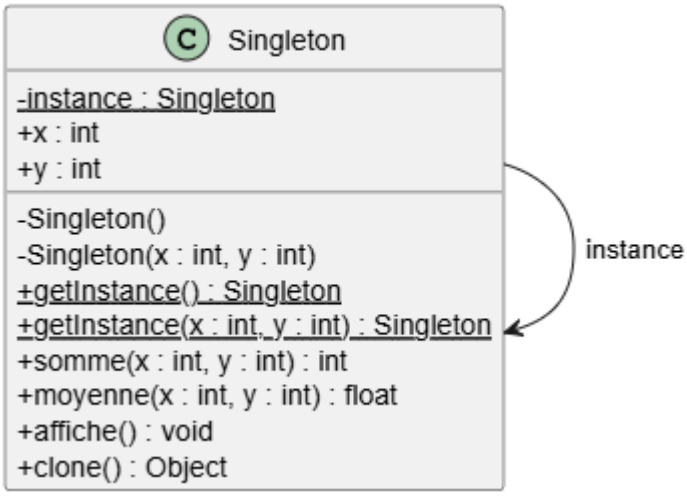



Figure 1 — Diagramme de classe de l'exemple du cours.

> **Comportement du test fourni :** `TestSingleton` appelle d'abord `getInstance()`, ce qui crée l'objet avec les valeurs par défaut (0, 0). Les appels suivants à `getInstance(8, 3)` et `getInstance(16, 19)` renvoient cette même instance ; les paramètres ne réinitialisent donc pas ses attributs.

# 3. Implémentation de l'exemple du cours

## Singleton.java

```
package cours;

public final class Singleton implements Cloneable {
    private static Singleton instance = null;
    public int x;
    public int y;

    private Singleton() {
        super();
    }

    private Singleton(int x2, int y2) {
        super();
        this.x = x2;
        this.y = y2;
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public static Singleton getInstance(int x, int y) {
        if (instance == null) {
            instance = new Singleton(x, y);
        }
        return instance;
    }

    public int somme(int x, int y) {
        return x + y;
    }

    public float moyenne(int x, int y) {
        return (float) somme(x, y) / 2;
    }

    public void affiche() {
        System.out.println("\nJe suis une instance et mes valeurs sont : x = " + this.x + " et y = " + this.y);
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException();
    }
}
```

## TestSingleton.java

```
package cours;

public class TestSingleton {
    public static void main(String[] args) {
        int som = Singleton.getInstance().somme(2, 5);
        System.out.println("La somme est : " + som);

        Singleton s1 = Singleton.getInstance(8, 3);
        s1.affiche();

        Singleton s2 = Singleton.getInstance(16, 19);
```

```
        s2.affiche();
    }
}
```

Le patron Singleton · Code source du cours

# 4. Exemple modifié : AppLogger

La variante remplace l'objet de calcul par `variante.AppLogger`, un journaliseur partagé. Chaque module récupère le même service via `AppLogger.getInstance()`, puis écrit des messages informatifs ou d'erreur horodatés. L'initialisation est paresseuse ; l'accesseur synchronisé protège la création si plusieurs threads y accèdent en même temps. Le clonage est refusé.

## Modélisation

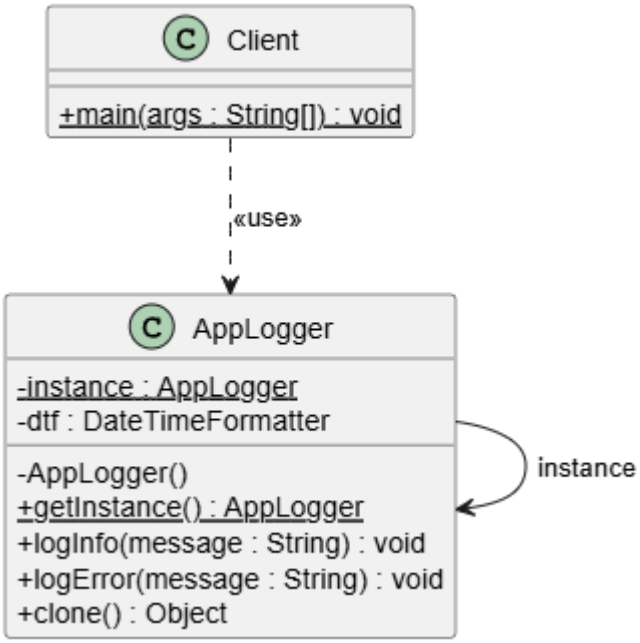



Figure 2 — Diagramme de classe de la variante AppLogger et de son client.

## Analyse du choix

Un journaliseur est un service transversal utilisé par plusieurs parties d'une application. L'unique instance représente ici un point de journalisation central, évitant la création d'instances distinctes par chaque composant. Le client effectue deux demandes d'instance et compare les références ; le résultat attendu est `true`. Cette démonstration écrit sur la sortie standard et la sortie d'erreur, sans ouvrir de fichier de logs.

# 5. Implémentation de la variante

## AppLogger.java

```
package variante;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public final class AppLogger {
    private static AppLogger instance = null;
    private DateTimeFormatter dtf;

    private AppLogger() {
        super();
        this.dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        System.out.println("[SYSTEM INIT] Initialisation du journal d'application unique (AppLogger).");
    }

    public static synchronized AppLogger getInstance() {
        if (instance == null) {
            instance = new AppLogger();
        }
        return instance;
    }

    public void logInfo(String message) {
        String timestamp = dtf.format(LocalDateTime.now());
        System.out.println("[INFO][" + timestamp + "] " + message);
    }

    public void logError(String message) {
        String timestamp = dtf.format(LocalDateTime.now());
        System.err.println("[ERROR][" + timestamp + "] " + message);
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException("Le Singleton AppLogger ne peut pas être cloné !");
    }
}
```

## Client.java

```
package variante;

public class Client {
    public static void main(String[] args) {
        AppLogger logger1 = AppLogger.getInstance();
        logger1.logInfo("Démarrage du module de sécurité.");

        AppLogger logger2 = AppLogger.getInstance();
        logger2.logInfo("Connexion au service de paiement établie.");

        System.out.println("\n--- Vérification de l'unicité ---");
        System.out.println("logger1 et logger2 pointent-ils vers la même instance ? " + (logger1 ==
logger2));
    }
}
```

Le patron Singleton · Code source de la variante